

Cloud Admin & Owner Task Inventory: LLDAP + OpenFGA + libcloud REST + Vault Architecture

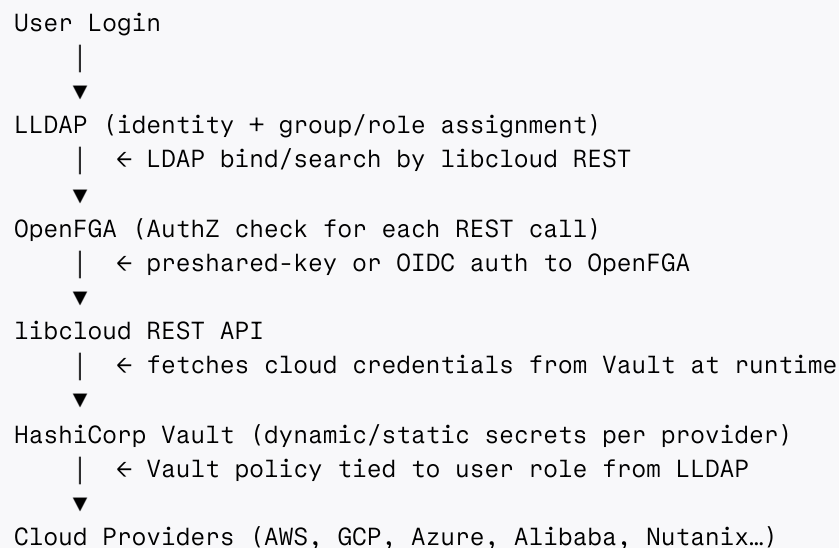
Overview

This document maps every recurring task that cloud administrators and cloud owners must perform within the described architecture, which comprises four interacting systems:

- **LLDAP** — lightweight LDAP server that acts as the identity source, issuing users and group/role memberships
- **OpenFGA** — fine-grained authorization server that controls access to every libcloud REST API endpoint
- **libcloud REST API** — multi-cloud provisioning abstraction layer that translates REST calls into provider-specific operations (AWS, GCP, Azure, Alibaba, Nutanix, etc.)
- **HashiCorp Vault** — secrets backend that stores per-provider credentials; Vault policies are bound to LLDAP-derived roles so that only authorised users can trigger provisioning that reaches the cloud

Tasks are segmented by **who** executes them (Cloud Owner vs Cloud Admin), **how** they are fulfilled (pre-scripted runbook vs on-demand developer work), and **which** system layer they touch.

System Interaction Map



Part 1 — Cloud Owner Tasks

The Cloud Owner has global administrative scope: they define the authorisation model, manage Vault root-level configuration, and govern the roles that everything else hangs off. Most of these tasks are **infrequent but high-impact**, and many should be pre-scripted as CLI runbooks.

1.1 Identity & Role Governance (LLDAP)

Task	Frequency	Fulfillment
Define LDAP group schema (group naming convention that maps to roles, e.g. <code>cloud-admin</code> , <code>cloud-owner</code> , <code>cloud-dev-aws</code> , <code>cloud-ro</code>)	One-off / per new provider	Developer pre-scripts LDAP schema; Owner approves
Create top-level groups / roles in LLDAP	Per new team or project	Pre-scripted: <code>lldap-cli group create</code> wrapper script
Assign users to groups (initial onboarding)	Per new hire / contractor	Pre-scripted onboarding runbook
Remove users from all groups (offboarding)	Per departure	Pre-scripted offboarding runbook (must also trigger Vault token revocation)
Rotate LLDAP admin credentials	Quarterly / as needed	Pre-scripted Vault-backed rotation script
Audit group memberships	Monthly	Pre-scripted report script querying LDAP memberOf

LLDAP exposes a GraphQL API and a web UI; group and user management can be automated via the GraphQL endpoint, making it straightforward to wrap in scripts.^{[1][2]}

1.2 Authorization Model Governance (OpenFGA)

OpenFGA authorization models are **immutable** — every change creates a new model version with a new model ID, which must be pinned in libcloud REST's configuration.^[^3]

Task	Frequency	Fulfillment
Define or revise the authorization model (which roles can call which REST endpoints, with what conditions)	Per new API surface / per policy change	Developer work — model written in FGA DSL, reviewed by Owner
Publish new model version and pin new <code>authorization_model_id</code> in libcloud REST config	Per model change	Developer + Owner: model PR → CI pipeline publishes to OpenFGA store → config updated
Write/delete relationship tuples when new users/roles become active (e.g. <code>user:alice</code> has <code>role:cloud-admin</code>)	Per onboarding/offboarding	Pre-scripted: triggered by LLDAP group-change webhook or reconciliation cron
Audit relationship tuples in the FGA store	Monthly / post-incident	Pre-scripted: <code>fga tuple read</code> sweep; results compared to LLDAP ground truth

Task	Frequency	Fulfillment
Shadow-check new model against production traffic before full rollout	Per model migration	Developer work — shadow check using dual model IDs ^[^3]
Rotate OpenFGA preshared keys	Quarterly	Pre-scripted: update key in Vault → rolling restart of libcloud REST → update OpenFGA config ^[4] ^[5]
Disable the OpenFGA playground on non-local environments	One-off at deployment	Infrastructure config (Terraform/Helm) — set <code>playground.enabled: false</code> ^[^6]

1.3 Vault Root Configuration (Secrets & Policy Governance)

Vault policies are the binding layer between LLDAP roles and cloud credentials.^[7]^[8]

Task	Frequency	Fulfillment
Define and write Vault policies per role (e.g. policy <code>cloud-admin-aws</code> allows <code>read</code> on <code>aws/creds/admin-role</code>)	Per new role or provider	Developer writes HCL policy; Owner approves and applies via <code>vault policy write</code>
Bind Vault policies to LDAP groups using the Vault LDAP auth method	Per new role	Pre-scripted: <code>vault write auth/ldap/groups/<group> policies=<policy></code>
Enable and configure cloud secrets engines (AWS, Azure, GCP, Alibaba) with root credentials	Per new provider onboarding	Developer work — one-off per provider; credentials stored out-of-band
Define Vault roles per cloud provider + permission level (e.g. <code>aws/roles/ec2-admin</code> , <code>aws/roles/ec2-readonly</code>)	Per new capability needed	Developer work; IAM policy documents authored per provider ^[^9]
Rotate root credentials for each cloud secrets engine	Quarterly	Pre-scripted: <code>vault write <engine>/config/rotate-root</code>
Revoke all leases for a departing user or compromised credential	Immediate / per incident	Pre-scripted: <code>vault lease revoke -prefix <path> /</code> ^[^10]
Audit Vault audit logs for anomalous secret access	Weekly / per alert	Pre-scripted: log pipeline to SIEM; Owner reviews anomaly dashboard
Unseal Vault after restart (if not using auto-unseal)	Per restart	Operational runbook with quorum of key holders

Part 2 — Cloud Admin Tasks

Cloud Admins operate within the policy envelope that the Owner has defined. Their daily work is provisioning, access management, and incident response — most of which should be exposed as pre-scripted commands that call libcloud REST, with Vault credential injection happening transparently.

2.1 User Lifecycle Management (LDAP-driven)

Task	Frequency	Fulfillment
Add a new user to LDAP (create account, set password)	Per onboarding	Pre-scripted onboarding script (LDAP GraphQL <code>createUser</code> mutation)
Assign user to one or more role-groups in LDAP	Per onboarding / role change	Pre-scripted: <code>ldap-cli group add-member</code> wrapper
Remove user from groups / disable account	Per offboarding or suspension	Pre-scripted offboarding runbook — must also delete OpenFGA tuples and revoke Vault tokens
Reset a user's LDAP password	On request	Pre-scripted self-service reset or admin-initiated via LDAP GraphQL
List users in a group (access review)	Monthly	Pre-scripted LDAP query / LDAP API report

2.2 Access Control — OpenFGA Tuple Management

Relationship tuples in OpenFGA represent the live state of "who can do what". Every time a user gains or loses a role, tuples must be written or deleted.^[11]

Task	Frequency	Fulfillment
Write access tuple when user joins a role (e.g. <code>user:alice → member of role:cloud-admin</code>)	Per onboarding / role change	Pre-scripted: triggered by LDAP group change; reconciliation job writes <code>WriteTuples</code> call to OpenFGA ^[11]
Delete access tuple when user leaves a role	Per offboarding / role change	Pre-scripted: same reconciliation job, calls <code>DeleteTuples</code>
Grant temporary elevated access (break-glass) for a specific resource	On demand	Pre-scripted: timed tuple write + scheduled deletion (developer builds the scheduler; admin invokes)
Verify access before a provisioning action (dry-run check)	On demand	Pre-scripted: <code>fga check user:<id> <relation> <object></code> wrapper
List all objects a user has access to	On demand / audit	Pre-scripted: <code>fga list-objects</code> for the user ^[11]
List all users who have access to a specific resource	Audit / periodic	Pre-scripted: <code>fga list-users</code> query

2.3 Cloud Provisioning via libcloud REST

These are the core day-to-day operational tasks — all routed through the libcloud REST API, with credentials sourced from Vault and authorization checked by OpenFGA on each call.^[12]
^[13]

Compute

Task	Frequency	Fulfillment
List available regions, instance sizes, and images for a provider	On demand	Pre-scripted: GET /compute/{provider}/sizes, /images
Provision a new VM / compute instance	On demand (user request)	Pre-scripted template-based script (size, image, region, SSH key as parameters)
Start / stop / reboot an existing node	On demand	Pre-scripted: POST /compute/{provider}/nodes/{id}/action
Destroy / delete a node	On demand	Pre-scripted: destructive action with confirmation prompt
List all running nodes and their state	Daily / on demand	Pre-scripted: GET /compute/{provider}/nodes
Resize / change instance type	On demand (user request)	Developer builds resize workflow (provider-specific steps vary)
Attach/detach storage volumes	On demand	Developer builds as provider-specific operations differ
Manage SSH key pairs (upload, list, delete)	Per project setup	Pre-scripted: POST /compute/{provider}/keypairs
Manage security groups / firewall rules	On demand	Developer work — security rules require review workflow

Storage

Task	Frequency	Fulfillment
Create / delete object storage buckets or block volumes	On demand	Pre-scripted with naming-convention enforcement
Upload / download objects	On demand	Pre-scripted or exposed via libcloud REST endpoints
List buckets and their metadata	Daily / on demand	Pre-scripted reporting script
Configure bucket lifecycle policies	Per project	Developer builds policy template runner

Networking (provider-dependent)

Task	Frequency	Fulfillment
List available VPCs, subnets, and zones	On demand	Pre-scripted: GET /network/{provider}/...
Allocate / release floating IPs	On demand	Pre-scripted
Manage load balancer instances	On demand	Developer work — LB config is complex and provider-specific
DNS record management	On demand	Developer work — DNS changes have broad impact, need review

2.4 Vault Credential Operations (Day-to-Day)

Admins interact with Vault indirectly (libcloud REST calls Vault internally), but some operational tasks are Admin-facing.^{[9][10]}

Task	Frequency	Fulfillment
Verify Vault is reachable and unsealed	Daily (health check)	Pre-scripted: <code>vault status</code> health probe in monitoring
Renew a long-lived token before expiry (if service tokens are used)	Before TTL expiry	Pre-scripted: automated renewal job (Vault Agent or cron) ^[10] ^[14]
Revoke a specific dynamic credential lease immediately after incident	Per incident	Pre-scripted: <code>vault lease revoke lease_id</code> ^[10]
Request a manual dynamic credential for a one-off operation	On demand	Pre-scripted: <code>vault read <engine>/creds/<role></code> – credential is time-bound
View active leases for a path	On demand / audit	Pre-scripted: <code>vault list sys/leases/lookup/<path></code> ^[15]
Rotate a static secret stored in Vault (e.g. API key for a SaaS integration)	Quarterly / on trigger	Pre-scripted rotation runbook

2.5 Observability & Incident Response

Task	Frequency	Fulfillment
Review OpenFGA authorization denial logs for unexpected blocks	Daily	Pre-scripted: log query / alert rule
Investigate why a user's API call was denied (trace through OpenFGA → LLDAP → Vault)	On demand (incident)	Pre-scripted diagnostic script: checks tuple existence → checks LLDAP group → checks Vault policy
Review Vault audit logs for anomalous credential access patterns	Weekly	Pre-scripted: log pipeline to SIEM
Check libcloud REST API error rates per provider	Daily	Pre-scripted: metrics dashboard (Prometheus/Grafana or similar)
Respond to a Vault lease expiry that caused a provisioning failure	On incident	Runbook: identify lease, renew or request new credential, retry operation ^[10]

Part 3 — Developer-Built Components (Non-Scriptable Demand Work)

The following tasks **cannot be pre-scripted** because they depend on user intent, new business requirements, or provider-specific complexity. Developers must build these as new features or workflows.

Capability	Reason Requires Developer	Systems Involved
New cloud provider onboarding	Requires: new libcloud driver integration, new Vault secrets engine config, new OpenFGA relations for provider-specific resources, new LLDAP groups	All four systems

Capability	Reason Requires Developer	Systems Involved
New resource type support (e.g. managed Kubernetes, serverless, databases)	libcloud REST must expose new endpoints; OpenFGA model must define new relations; Vault may need new dynamic secret roles	libcloud REST + OpenFGA + Vault
Multi-cloud topology provisioning (VMs across providers in one request)	Requires orchestration logic, distributed rollback, cross-provider dependency resolution	libcloud REST (new composite endpoint)
Budget and quota enforcement per team/role	Requires a new policy layer querying cost APIs + blocking provisioning if threshold exceeded	libcloud REST + OpenFGA condition model
Automated resource expiry / TTL-based deprovisioning	Requires scheduler, state tracking, grace-period notification, and deprovisioning workflow	libcloud REST + new state store
Self-service provisioning portal / UI	Full frontend + backend work; inputs become libcloud REST API calls	All systems
Change approval workflow (request → approve → provision)	Requires workflow engine, notification integration, audit trail	libcloud REST + OpenFGA (approval relation)
Cost tagging and FinOps metadata enforcement	Provider-specific tag injection at create time; tag policy engine	libcloud REST
Audit export and compliance reporting	Aggregation of LLDAP audit, OpenFGA logs, Vault audit logs, and cloud provider CloudTrail/Activity logs into a unified report	Cross-cutting
OpenFGA model migration for a breaking change	Requires shadow-check harness, tuple migration script, progressive rollout with dual model IDs[^3]	OpenFGA + libcloud REST config
Vault dynamic secrets for a new provider (e.g. Alibaba, Nutanix)	Provider-specific Vault plugin or custom secrets engine may need development	Vault + libcloud REST
Break-glass access with time-bound escalation	Timed tuple write + auto-delete in OpenFGA; Vault policy elevation + automatic revocation[^10]	OpenFGA + Vault

Part 4 — Synchronisation & Consistency Concerns

Because identity lives in LLDAP, authorization state lives in OpenFGA, and access policy lives in Vault, the systems can drift. The following reconciliation jobs should be built and scheduled by developers, then monitored by admins.

Job	Purpose	Trigger
LLDAP → OpenFGA reconciler	Compare LLDAP group memberships to OpenFGA tuples; write missing tuples, delete stale ones	Scheduled cron (e.g. every 5 min) + on LLDAP group-change event
LLDAP → Vault LDAP auth sync	Ensure Vault's <code>auth/ldap/groups</code> mappings match current LLDAP groups	Scheduled cron; alert on mismatch
OpenFGA stale tuple sweep	Identify tuples referencing users or roles that no longer exist in LLDAP	Daily cron

Job	Purpose	Trigger
Vault lease expiry monitoring	Alert before critical leases expire; trigger renewal or notify dependent service	Vault Agent or external monitor ^[10] [14]
Dead-letter queue for failed provisioning	Capture libcloud REST calls that failed due to auth errors (FGA deny, Vault lease expiry) for admin review	libcloud REST middleware

Part 5 — Role-Responsibility Matrix

Task Category	Cloud Owner	Cloud Admin	Developer (pre-build)
LLDAP schema & group design	✓ Approves	✓ Executes	✓ Scripts
User onboarding/offboarding	✓ Governs	✓ Executes	✓ Scripts
OpenFGA model design	✓ Approves	—	✓ Builds
OpenFGA tuple management	✓ Governs	✓ Executes	✓ Scripts
Vault policy authoring	✓ Approves	—	✓ Builds
Vault LDAP group binding	✓ Executes	—	✓ Scripts
Cloud provider secrets engine setup	✓ Executes	—	✓ Builds
Vault credential revocation	✓ Executes	✓ Executes (scoped)	✓ Scripts
Cloud provisioning (compute/storage/network)	—	✓ Executes	✓ Scripts
Observability & log review	✓ Governs	✓ Executes	✓ Builds dashboards
New provider / resource type onboarding	✓ Approves	—	✓ Builds entirely
Reconciliation jobs	✓ Governs	✓ Monitors	✓ Builds
Break-glass access	✓ Approves	✓ Invokes	✓ Builds mechanism

References

1. [lldap/example_configs/nexus.md at main · lldap/lldap](#) - Light LDAP implementation. Contribute to lldap/lldap development by creating an account on GitHub.
2. [How Group Roles Apply to Active Directory, LDAP](#) - When an administrator changes the role-based access control (RBAC) rules for a group, those revised ...
3. [Immutable Authorization Models | OpenFGA](#) - Update the authorization model with the renamed relation. A new model ID will be generated but it wo...
4. [Configuring OpenFGA](#) - If using Pre-shared key authentication, you will configure OpenFGA with one or more secret keys and ...
5. [OpenFGA Configuration Options](#) - Configuring Options for the OpenFGA Server
6. [OpenFGA Playground Preshared Key Exposure · Advisory - GitHub](#) - When OpenFGA is configured to use preshared-key authentication with the built-in playground enabled,...

7. [Policies in Vault - HashiCorp](#) - In this guide, you'll learn how to use policies in Vault, which control access privileges and author...
8. [Policies](#) - Policies provide a declarative way to grant or forbid access to certain paths and operations in Vault...
9. [Cloud Dynamic Secrets with HashiCorp Vault - Joshua Powers](#) - This is the sixth post in a series on Vault. For previous posts see, [Getting Started](#), [Secrets Engine](#)...
10. [Lease, Renew, and Revoke | Vault](#) - With every dynamic secret and service type authentication token, Vault creates a lease: metadata con...
11. [Managing Relationships Between Objects | OpenFGA](#) - Granting a user access to a particular object through a relationship with another object
12. [Using apache-libcloud to provision cloud servers - Pixelite](#) - Apache's libcloud is a Python library that allows you to write code that interacts with numerous clo...
13. [Using Apache Libcloud for declarative and procedural multi...](#) - Apache Libcloud is a Python library which hides differences between different cloud provider APIs an...
14. [Revoking Leases](#) - Master HashiCorp Vault lease management and automatic renewal strategies in Kubernetes to maintain c...
15. [Manage dynamic credential leases | Vault](#) - Manage leases for Vault dynamic credentials.